

Day2 · 8차시

에이전트 설계 & 사용자 경험

한 번 돌리기는 쉽다. 항상 잘 돌게 설계하는 게 어렵다.
7차시에서 돌린 루프를 — 이제 신뢰할 수 있게 만든다.

SECTION 00

오프닝 — 되긴 된다 vs 잘 된다

설계와 UX는 동전의 양면: 잘 만들고, 그걸 신뢰하게 만든다.

데모는 한 번, 프로덕션은 매번

7차시: 되긴 된다

- 추론+도구+루프로 ReAct를 돌렸다
- 한 번 돌면 '오 되네'
- 입력을 내가 골라서 줬다
- 데모의 세계

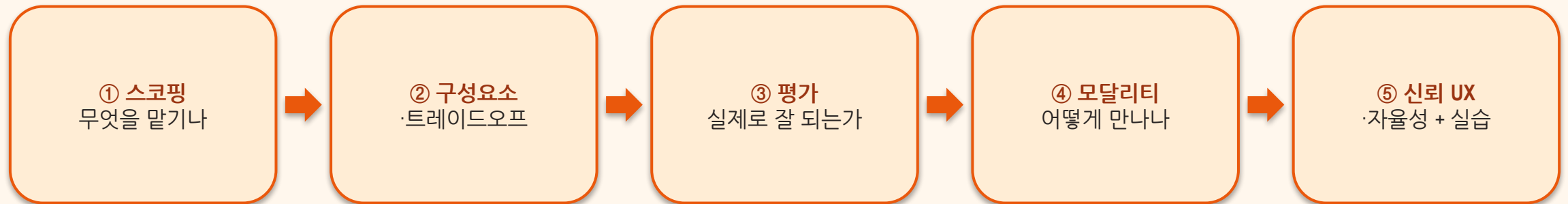
VS

8차시: 잘 된다

- 100번 중 몇 번 맞나?
- 모델은 확률적 — 한 번 맞음 ≠ 항상 맞음
- 무엇을 말기고 무엇을 안 말길지
- 프로덕션의 세계

이 차시 동안 얻는 것

▶ 앞 절반은 설계(Ch.2), 뒤 절반은 UX(Ch.3) + 실습



SECTION 01

스코핑 — 무엇을 말기나

설계는 지저분하게 시작한다

▶ 거창한 설계서로 시작하는 사람은 없다

① 지저분한 진짜 문제
(고객 메일 매일 수백 건)



② API 키 하나

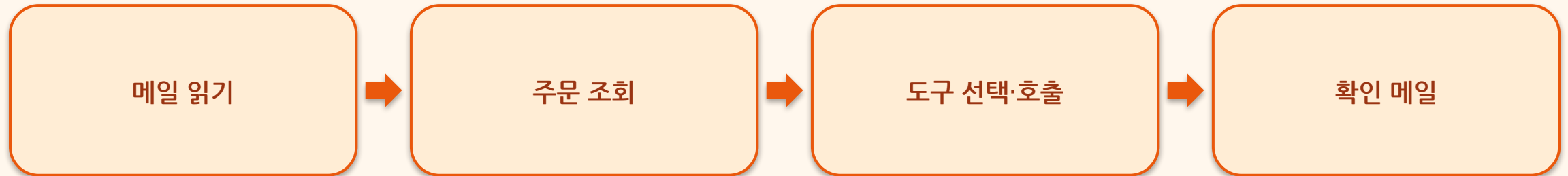


③ 막연한 아이디어
(‘이러면 도움될 것 같다’)

🔗 이론을 완성한 뒤가 아니라 — 일단 동작시키며 배운다

사람이 하던 2분짜리 반복

▶ issue_refund · cancel_order · update_address 중 하나를 올바른 파라미터로 호출



좁다 · 넓다 · 모호하다

▶ 스코핑은 늘 균형 잡기(balancing act) — 막연하면 가장 좁고 측정 가능하게

너무 좁게

'최소만' 처리
물량 많은 다른 요청 놓침
→ 임팩트가 작다

너무 넓게

'다 자동화'
청구분쟁·추천·기술지원...
→ 엣지케이스 범람

너무 모호

'고객 만족도 향상'
성공 기준이 없음
→ 잘했는지 알 수 없다

좋은 스코프의 조건 — '주문 #B73973 취소'

1

구체적 입력

고객 메시지 + 주문 레코드 (모호한 의도가 아니라 명확한 데이터)

2

구조화된 출력

`cancel_order(order_id="B73973")` + 확인 메시지

3

촘촘한 피드백 루프

잘했는지/못했는지 빠르게 확인 가능

핵심: 사람의 절차가 곧 도구 호출로 1:1 번역 — 조회→확인→'취소'→답장 = 도구 한 번 + 확인 한 줄

SECTION 02

핵심 구성요소 — 무엇으로 만드나

에이전트 시스템 — 오케스트레이션이 조율한다

▶ 개별이 아니라 함께 맞물려 동작해야 — 조율이 없으면 부품 더미



대형 모델 vs 소형 모델

▶ '작다고 늘 못한 건 아니다' — 실시간·자원제약이면 소형이 실용적

대형 (GPT-5·Claude Opus)

- 열린·다단계·창의 과제
- 미묘한 이해·유연한 추론
- 개인비서·리서치·기업용
- 느리고 비쌈 → 아껴서 배치

VS

소형 (Phi-4·증류모델)

- 정형·반복 과제
- 정밀도 필요, 창의성 덜
- 고객지원·검색·라벨링
- 빠르고 저렴 → 로컬 가능

무엇으로 고르나

▶ 한 번 정하면 끝이 아니라 — 코드처럼 계속 재검토하는 유지보수 대상

1

작업 복잡도

열린·다단계면 대형, 정형·반복이면 소형

2

모달리티

이미지·음성을 정말 다뤄야 하나? 아니면 텍스트 전용이 더 빠르다

3

개방성

오픈소스(투명·파인튜닝·프라이버시) vs API(편의·속도·관리형)

4

사전학습 vs 커스텀

실수 비용 큰 전문영역(의료·법률)이면 맞춤 학습

5

비용·지연

현실 배포의 결정타 — 하이브리드·동적 라우팅(Dynamic Model Routing)

비싼 모델 = 더 똑똑한 모델? (X)

▶ 기준은 절대 성능이 아니라 성능 대비 가격(price per performance)

가정

- '제일 비싼 걸 쓰면 제일 잘하겠지'
- 모든 요청에 플래그십

VS

현실 (MMLU 기준)

- DeepSeek-v3: 최고 성능인데 상대가 저렴
- 가장 비싼 모델이 점수는 더 낮기도
- 벤치마크는 참고용 — 내 작업으로 직접 비교
- 쉬운 일은 싼 모델, 어려운 일만 비싼 모델

네 축을 동시에 최대화할 수 없다

▶ 스코프가 알려준다 — 내 서비스에서 어느 축이 진짜 중요한가

성능
속도·정확도
응답 품질

확장성
부하 증가 대응
지연 최소화

신뢰성
내결함성·일관성
테스트 가능성

비용
토큰·인프라
운영비

모든 설계 선택은 이 네 축 사이의 거래 — 어느 축을 내줄지 '먼저' 정하는 것이 설계다

단일 에이전트 vs 멀티 에이전트

▶ 복잡도가 필요를 증명할 때까지 단일로 — 멀티는 12차시에서

단일 에이전트

- 단순 — 만들기·디버깅 쉬움
- 잘 정의된 작업에 충분
- 토큰·비용 예측 가능
- 우리 팀 프로젝트는 여기서 시작

VS

멀티 에이전트

- 협업·병렬 처리·확장성
- 역할 분담 (리서처+작성자...)
- 조율 복잡도·토큰 소비 급증
- 수명 긴 대형 시스템용

SECTION 03

평가 — 실제로 잘 되는가

테스트되지 않은 에이전트는 신뢰할 수 없는 에이전트다.

“An untested agent is an untrusted agent.”

테스트되지 않은 에이전트는 신뢰할 수 없다.

'실행됨' ≠ '잘 됨' — 취소 에이전트 점검

1

올바른 도구를 골랐나

tool precision/recall — cancel_order를 골랐는가

2

올바른 파라미터인가

parameter accuracy — 정확한 주문 ID를 넘겼는가

3

확인 메시지가 명확·정확한가

고객에게 '취소됨'을 분명히 전했다는가

4

다양한 예시로 성공률 측정

배포 전 엡지케이스를 미리 잡는다

assert 두 줄로 시작한다

▶ 입력을 넣고 → 도구가 불렀나? 확인 문구가 있나? → 통과/실패

```
result = run_agent("주문 #B73973 취소해줘. 더 싼 걸 찾았어.")  
  
assert "cancel" in result.tools_used, "도구 미호출"  
assert "취소" in result.reply, "확인 메시지 없음"  
print("✅ 취소 평가 통과")
```

최소 → 본격: 무엇을 어떻게 재나

▶ 결국 앞의 세 질문(도구·파라미터·응답)을 비율로 바꾼 것

도구 정밀도

precision — 호출한 도구가 실제로 맞았던 비율

도구 재현율

recall — 불러야 할 도구를 빠뜨리지 않은 비율

파라미터 정확도

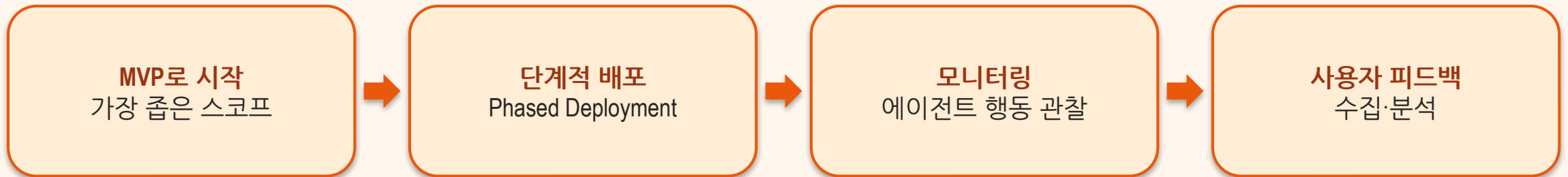
넘긴 인자(주문 ID 등)가 정확했던 비율

작업 성공률

다양한 케이스 배치 평가 → 끝까지 완수된 비율

반복적 설계(Iterative Design) — 배포는 한 방이 아니다

▶ 조기 발견 · 사용자 중심 · 통제된 성장 — 책의 모범 사례 3종 세트



🔄 Iterate — 통찰을 다음 버전에 반영, 필요할 때만 스코프 확장

SECTION 04

상호작용 모달리티 — 사용자는 어떻게 만나나

여기서부터 UX. 잘 만든 것을 잘 쓰게.

무엇으로 대화하나

▶ 선택이 사용자의 인식 방식을 결정 — 읽기 250~300wpm > 듣기 150~180wpm

텍스트

명료·추적·비동기
챗봇·CLI·생산성
기록 남음 → 디버깅

GUI

시각 구조·인지부하↓
대시보드·AI IDE
멀티스텝·상태표시

음성

핸즈프리·자연 대화
Siri·콜센터
운전·요리 중

생성형 UI

상황 맞춤 화면을
그때그때 생성
품질·일관성 관리 필요

강점은 분명하다 — 함정도 분명하다

▶ 기능이 풍부해도 존재를 모르면 없는 것과 같다

강점

- 단순·친숙·통합 쉬움
- 동기/비동기 모두
- 추적 가능한 기록 — 투명·책임·디버깅
- 터미널의 르네상스(Warp·Claude Code)

VS

함정: 발견가능성

- GUI는 버튼·메뉴로 어포던스 제공
- 텍스트는 빈 입력창뿐
- 뭘 할 수 있는지 추측해야
- 지원 안 하면 불투명한 거절만

동기 vs 비동기 경험

▶ 오래 걸리는 작업이 생기는 순간 — 진행 표시와 알림이 신뢰를 만든다

동기 — 실시간 대화

- 즉각적 주고받기·턴테이킹
- 타이핑 인디케이터 같은 시각 단서
- 명료·간결이 생명
- 챗봇·음성 비서

VS

비동기 — 맡겨두는 작업

- 진행 상태 표시(Progress Indicator)
- 명확한 타임라인·소요 시간 예상
- 완료·문제 시 후속 알림
- 리서치·배치 작업 에이전트

SECTION 05

신뢰를 주는 UX — 신뢰는 설계된다

무엇이 신뢰를 만드나

▶ 맥락 관리가 없으면 아무리 진보한 에이전트도 단절되고 반응 없는 느낌

① 컨텍스트 관리

복불 시키지 말 것
Cursor처럼 환경에서
관련 정보를 자동으로

② 역량·한계 전달

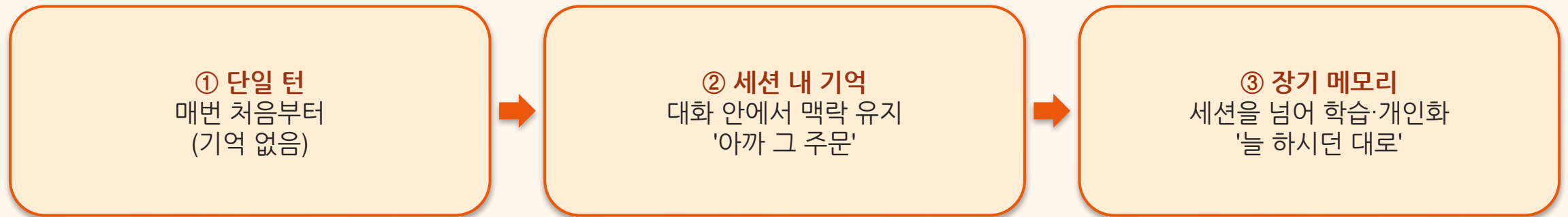
무엇을 할 수 있고/없고
언제 사람이 개입해야
하는지 알린다

③ 투명성

예측 가능한 행동 +
행동에 대한
명확한 설명

맥락의 3수준 — 갈수록 '나를 아는' 에이전트

▶ 수준이 오를수록 신뢰도, 책임도 커진다



빈 안내 vs 범위 명시

▶ 점진적 공개(Progressive Disclosure) — 처음엔 핵심만, 익숙해지면 고급 기능

이렇게 말고 (X)

- “무엇을 도와드릴까요?”
- 빈 입력창 + 침묵
- 사용자가 찢어보며 시행착오

VS

이렇게 (O)

- “주문 취소·배송 조회·계정 변경을 도와드려요”
- 온보딩·주기적 안내·동적 제안
- 운영 경계를 먼저 알린다

The Autonomy Slider — Manual · Ask · Agent

▶ 자율성은 고정이 아니라, 신뢰·역량·맥락에 따라 옮겨가는 손잡이



Manual (수동)
사람이 전부 직접
에이전트는 구경만

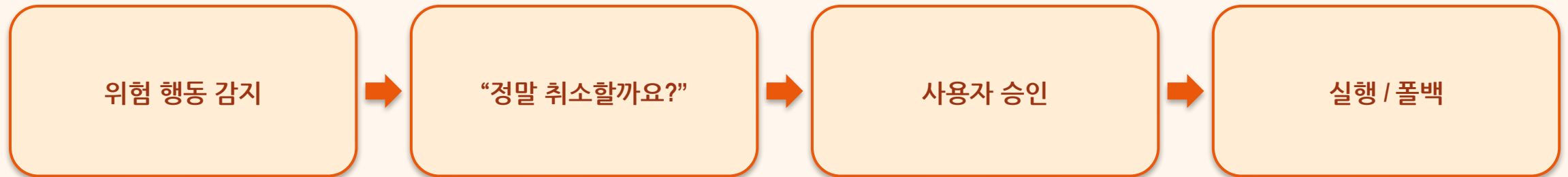
Ask (보조) ← 지금 여기
에이전트가 제안
사람이 검토·승인

Agent (자율)
표준 작업은 알아서
예외만 사람에게

신뢰가 쌓일수록 오른쪽으로 — 단계를 명확히 노출하고, 전환은 매끄럽게

불확실성 · 확인 · 되돌리기

▶ 확신 낮으면 그렇다고 표시 · 비가역 작업 전 확인 · 막다른 거절 대신 대안·롤백



☞ 실패 시 — 우아한 실패 + 사람에게 에스컬레이션

Failing Gracefully — 실패도 신뢰의 기회

▶ 막다른 거절이 아니라, 다음 길을 열어주는 것

① 상태 보존

State Preservation
진행 상황을 잃지 않는다
처음부터 다시 = 최악

② 공감의 언어

사과 + 무슨 일이
있었는지 솔직한 설명

③ 해결 경로

대안 제시·문제 해결 단계
사람에게 에스컬레이션

신뢰를 깨는 안티패턴 모음

▶ 각 실수에 오늘 배운 원칙이 하나씩 짝지어져 있다

막연한 스코프

'다 자동화' → 좁게·측정 가능하게

불투명한 거절

이유·대안 없는 'X' → 우아한 거절 + 안내

무한 루프

종료 조건 없음 → max_steps 한도

확인 없는 비가역 작업

삭제·결제 즉시 실행 → 승인 후 실행

평가 없이 배포

'한 번 됐으니 됐다' → 평가 내장

SECTION 06

[실습] 내 에이전트를 설계·평가

실습/8차시_실습.ipynb — 스코핑 → 구현 → 평가 (약 20분)

4단계로

▶ 체크포인트: 평가 3케이스가 pass/fail로 찍히는가

1

스코핑 1장

하는 것 / 안 하는 것 — 코드이자 곧 '설계서'

2

범위를 시스템 프롬프트에

스코프 밖 요청은 우아하게 거절 + 대안 안내

3

평가 케이스 3개

입력·기대조건·설명 — assert로 pass/fail 확인

4

내 팀 MVP에 적용

우리 서비스의 스코핑 doc + 평가 케이스 작성

오늘 3줄

- 좁게 스코핑 → 모델·도구·메모리로 구성 → 평가 내장 → 모달리티 → 신뢰 UX
- '테스트되지 않은 에이전트는 신뢰할 수 없다' — 평가는 처음부터
- 신뢰는 우연이 아니라 설계된다 — 발견가능성·자율성 슬라이더·확인·되돌리기

다음 차시

Day2 로드맵에서 지금

▶ 다음 9차시 — 도구·MCP: 에이전트의 손발을 넓힌다



14 [캡스톤] AI 마이노트 개발기 — 오늘 정한 스코프·평가가 그대로 팀 프로젝트의 뼈대가 된다